KON429E Autonomous Vehicles

Homework 2: Extended Kalman Filter Implementation for 2D Motion Estimation

Atakan Kuşçu
040210520
Istanbul Technical University
Control and Automation Engineering

April 30, 2026

Abstract

This report presents the implementation and analysis of an Extended Kalman Filter (EKF) for two-dimensional vehicle motion estimation. The vehicle is modeled using a nonlinear kinematic bicycle model, where the state vector consists of the planar position coordinates and the heading angle. Noisy GPS-like measurements are available only for the position components, while the heading angle is estimated indirectly through the motion model and covariance coupling. The EKF is implemented in Simulink using two separate subsystems: a prediction block and an update block. The resulting position and heading estimates are compared with the ground-truth trajectory and noisy measurements. In addition, uncertainty ellipses are generated from the estimated covariance matrix, and the effect of increasing the measurement noise covariance matrix is investigated.

Contents

1 Introduction **4**

2 System Model **4**

2.1 State Vector 4

2.2 Nonlinear Kinematic Bicycle Model 5

2.3 Measurement Model 5

3 Extended Kalman Filter Derivation **5**

3.1 Prediction Step 5

3.2 Jacobian of the Motion Model 6

3.3 Update Step 6

3.4 Matrix Dimensions 7

4 Simulink Implementation **8**

4.1 Implementation Structure 8

4.2 Feedback Structure 9

4.3 Simulation Settings 9

5 Results **10**

5.1 Trajectory Estimation 10

5.2 Uncertainty Ellipses 11

5.3 Heading Angle Estimation 12

5.4 Position Error 13

5.5 Heading Error 13

5.6 Performance Metrics 14

6 Effect of Increasing Measurement Noise Covariance **15**

6.1 Motivation 15

6.2 Theoretical Effect of Increasing R 15

6.3 Comparison Results 15

7 Discussion **17**

8 Conclusion **18**

A MATLAB Initialization Script **18**

B Prediction Block MATLAB Function **19**

C Update Block MATLAB Function **20**

D Post-Processing Script **21**

E Measurement Noise Covariance Comparison Script **25**

1 Introduction

State estimation is a fundamental component of autonomous vehicle systems. Since real sensors are affected by noise, bias, and limited measurement availability, the vehicle state cannot usually be obtained directly with perfect accuracy. Instead, estimation algorithms are required to combine a mathematical process model with noisy sensor measurements.

In this homework, an Extended Kalman Filter (EKF) is implemented for estimating the planar motion of a vehicle moving in two dimensions. The nonlinear system model is based on the kinematic bicycle model. The vehicle state consists of the longitudinal position, lateral position, and heading angle. The available measurement is a GPS-like noisy position measurement, which provides only the x and y coordinates. Therefore, the heading angle is not directly measured and must be inferred from the vehicle motion model.

The main objectives of this work are:

- to derive the EKF prediction and update equations for the given nonlinear vehicle model,
- to implement the EKF in Simulink using separate prediction and update subsystems,
- to compare the EKF estimate with the ground-truth trajectory and noisy measurements,
- to visualize the covariance evolution using uncertainty ellipses,
- and to analyze the influence of increasing the measurement noise covariance matrix R.

2 System Model

2.1 State Vector

The state vector is defined as

$$\boldsymbol{x}_k = \begin{bmatrix} p_{x,k} \\ p_{y,k} \\ \theta_k \end{bmatrix}, \tag{1}$$

where $p_{x,k}$ and $p_{y,k}$ denote the vehicle position coordinates at time step k, and θ_k denotes the heading angle.

The system input vector is

$$\boldsymbol{u}_k = \begin{bmatrix} \delta_k \\ v_k \end{bmatrix}, \tag{2}$$

where δ_k is the steering angle and v_k is the vehicle speed.

2.2 Nonlinear Kinematic Bicycle Model

The discrete-time nonlinear motion model is given by

$$\boldsymbol{x}_k = f(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k) + \boldsymbol{w}_k, \tag{3}$$

where $\boldsymbol{w}_k$ is the process noise. The nonlinear state transition function is

$$f(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k) = \begin{bmatrix} p_{x,k-1} + v_k \cos(\theta_{k-1})\Delta t \\ p_{y,k-1} + v_k \sin(\theta_{k-1})\Delta t \\ \theta_{k-1} + \dfrac{v_k}{L} \tan(\delta_k)\Delta t \end{bmatrix}, \tag{4}$$

where L is the wheelbase and Δt is the sampling time.

This model is nonlinear because of the trigonometric terms $\cos(\theta)$, $\sin(\theta)$, and $\tan(\delta)$. Therefore, a standard linear Kalman filter cannot be directly applied. Instead, the Extended Kalman Filter is used by linearizing the nonlinear model around the current estimate.

2.3 Measurement Model

The measurement vector contains only the planar position coordinates:

$$\boldsymbol{z}_k = \begin{bmatrix} z_{x,k} \\ z_{y,k} \end{bmatrix}. \tag{5}$$

The measurement model is

$$\boldsymbol{z}_k = h(\boldsymbol{x}_k) + \boldsymbol{r}_k, \tag{6}$$

where $\boldsymbol{r}_k$ is the measurement noise. Since only position is measured,

$$h(\boldsymbol{x}_k) = \begin{bmatrix} p_{x,k} \\ p_{y,k} \end{bmatrix}. \tag{7}$$

The heading angle θ_k is not directly measured.

3 Extended Kalman Filter Derivation

3.1 Prediction Step

The EKF prediction step propagates the previous state estimate through the nonlinear motion model:

$$\hat{\boldsymbol{x}}_k^- = f(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{u}_k). \tag{8}$$

Using the kinematic bicycle model, this becomes

$$\hat{\boldsymbol{x}}_k^- = \begin{bmatrix} \hat{p}_{x,k-1} + v_k \cos(\hat{\theta}_{k-1})\Delta t \\ \hat{p}_{y,k-1} + v_k \sin(\hat{\theta}_{k-1})\Delta t \\ \hat{\theta}_{k-1} + \dfrac{v_k}{L} \tan(\delta_k)\Delta t \end{bmatrix}. \tag{9}$$

The covariance matrix is propagated as

$$P_k^- = F_k P_{k-1} F_k^T + Q, \tag{10}$$

where P_k^- is the predicted covariance, P_{k-1} is the previous updated covariance, and Q is the process noise covariance matrix.

3.2 Jacobian of the Motion Model

The EKF requires linearization of the nonlinear process model. Therefore, the Jacobian matrix F_k is calculated as

$$F_k = \left. \frac{\partial f}{\partial \boldsymbol{x}} \right|_{\boldsymbol{x}=\hat{\boldsymbol{x}}_{k-1}}. \tag{11}$$

From Equation (4), the analytical Jacobian is

$$F_k = \begin{bmatrix} 1 & 0 & -v_k \sin(\hat{\theta}_{k-1})\Delta t \\ 0 & 1 & v_k \cos(\hat{\theta}_{k-1})\Delta t \\ 0 & 0 & 1 \end{bmatrix}. \tag{12}$$

This matrix changes at every time step because it depends on the current heading estimate $\hat{\theta}_{k-1}$. This time-varying linearization is one of the key differences between the EKF and the standard Kalman filter.

3.3 Update Step

The measurement prediction is obtained as

$$\hat{\boldsymbol{z}}_k = h(\hat{\boldsymbol{x}}_k^-). \tag{13}$$

For the given position measurement model,

$$\hat{\boldsymbol{z}}_k = \begin{bmatrix} \hat{p}_{x,k}^- \\ \hat{p}_{y,k}^- \end{bmatrix}. \tag{14}$$

The innovation, or measurement residual, is

$$\boldsymbol{y}_k = \boldsymbol{z}_k - \hat{\boldsymbol{z}}_k. \tag{15}$$

The measurement Jacobian is

$$H_k = \left.\frac{\partial h}{\partial \boldsymbol{x}}\right|_{\boldsymbol{x}=\hat{\boldsymbol{x}}_k^-} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}. \tag{16}$$

Unlike F_k, the matrix H_k is constant because the measurement model is linear.
The innovation covariance is

$$S_k = H_k P_k^- H_k^T + R, \tag{17}$$

where R is the measurement noise covariance matrix.
The Kalman gain is computed as

$$K_k = P_k^- H_k^T S_k^{-1}. \tag{18}$$

The updated state estimate is

$$\hat{\boldsymbol{x}}_k = \hat{\boldsymbol{x}}_k^- + K_k \boldsymbol{y}_k. \tag{19}$$

Finally, the covariance update is

$$P_k = (I - K_k H_k) P_k^-. \tag{20}$$

3.4 Matrix Dimensions

The dimensions of the main EKF variables are summarized in Table 1.

Table 1: Dimensions of EKF variables.

Symbol	Dimension	Description
$\boldsymbol{x}_k$	3×1	State vector
$\boldsymbol{u}_k$	2×1	Control input vector
$\boldsymbol{z}_k$	2×1	Measurement vector
P_k	3×3	State covariance matrix
Q	3×3	Process noise covariance matrix
R	2×2	Measurement noise covariance matrix
F_k	3×3	Process model Jacobian
H_k	2×3	Measurement model Jacobian
S_k	2×2	Innovation covariance matrix
K_k	3×2	Kalman gain

4 Simulink Implementation

4.1 Implementation Structure

The EKF was implemented in Simulink using two clearly separated subsystems:

- **Prediction Block**
- **Update Block**

The prediction block receives the previous state estimate $\hat{\boldsymbol{x}}_{k-1}$, the previous covariance matrix P_{k-1}, and the current control input $\boldsymbol{u}_k$. It outputs the predicted state estimate $\hat{\boldsymbol{x}}_k^-$ and predicted covariance matrix P_k^-.

The update block receives the predicted state, predicted covariance, current measurement $\boldsymbol{z}_k$, and measurement noise covariance R. It outputs the corrected state estimate $\hat{\boldsymbol{x}}_k$ and corrected covariance matrix P_k.

The overall Simulink architecture is shown in Figure 1.

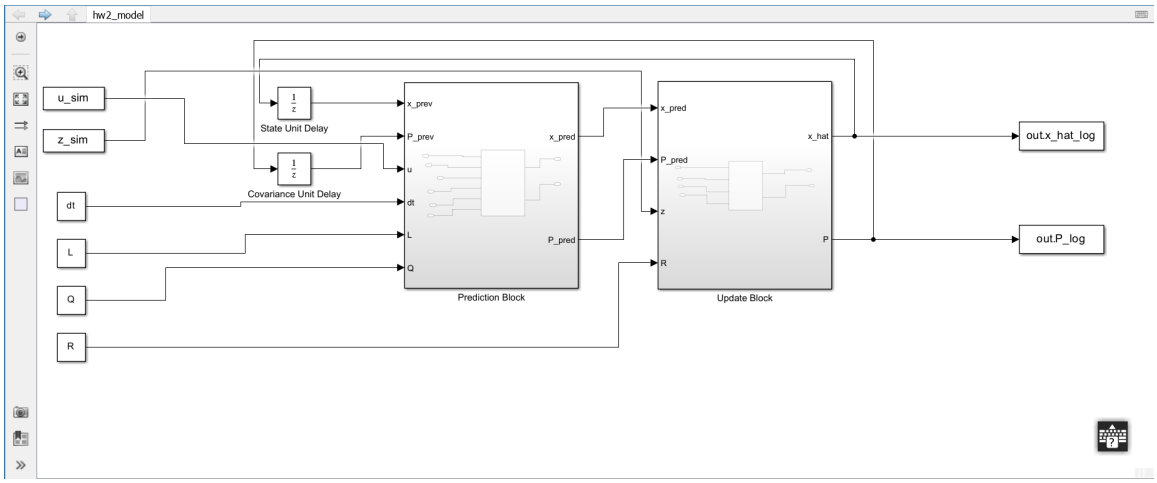


Figure 1: Overall Simulink model structure showing the Prediction Block and Update Block.

4.2 Feedback Structure

The EKF is a recursive estimator. Therefore, the updated values $\hat{\boldsymbol{x}}_k$ and P_k must be fed back as the previous values for the next time step. In the Simulink model, this was implemented using Unit Delay blocks.

The initial state estimate and covariance were selected as

$$\hat{\boldsymbol{x}}_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \tag{21}$$

and

$$P_0 = \begin{bmatrix} 50 & 0 & 0 \\ 0 & 50 & 0 \\ 0 & 0 & 50 \end{bmatrix}. \tag{22}$$

The relatively large diagonal entries of P_0 indicate high initial uncertainty in the state estimate.

4.3 Simulation Settings

The Simulink solver was set to fixed-step discrete simulation. The fixed-step size was selected equal to the provided sampling time:

$$\Delta t = 0.1 \text{ s}. \tag{23}$$

The total simulation duration was

$$T = 10 \text{ s}. \tag{24}$$

The wheelbase and constant vehicle speed were

$$L = 1.5 \text{ m}, \qquad v = 5 \text{ m/s}. \tag{25}$$

The process noise covariance matrix was

$$Q = \begin{bmatrix} 0.01 & 0 & 0 \\ 0 & 0.01 & 0 \\ 0 & 0 & 0.001 \end{bmatrix}, \tag{26}$$

and the nominal measurement noise covariance matrix was

$$R = \begin{bmatrix} 1.5 & 0 \\ 0 & 1.5 \end{bmatrix}. \tag{27}$$

5 Results

5.1 Trajectory Estimation

The estimated trajectory was compared with the ground-truth trajectory and the noisy GPS-like measurements. The result is shown in Figure 2.

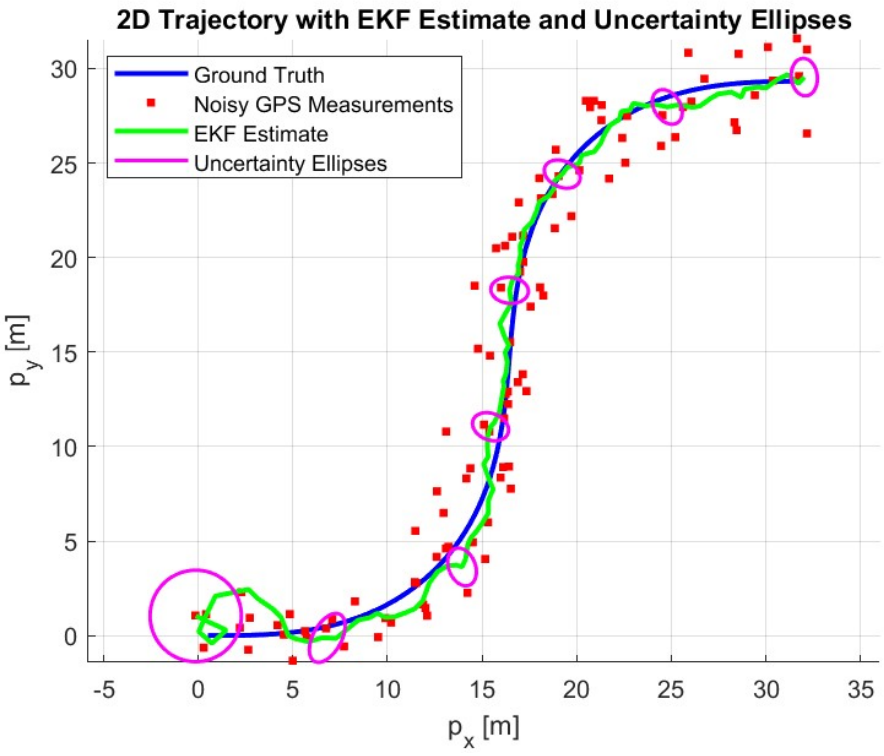


Figure 2: 2D trajectory comparison between ground truth, noisy GPS measurements, and EKF estimate with uncertainty ellipses.

The EKF estimate follows the ground-truth trajectory closely while filtering out the high-frequency noise present in the GPS-like measurements. The red measurement points are visibly scattered around the true trajectory, whereas the EKF estimate is smoother and remains close to the true path.

At the beginning of the simulation, a short transient behavior is observed due to the initial state uncertainty. After several measurement updates, the EKF estimate converges toward the ground truth and tracks the trajectory accurately.

5.2 Uncertainty Ellipses

The uncertainty ellipses in Figure 2 were generated using the 2×2 position covariance block of P_k:

$$P_{pos,k} = \begin{bmatrix} P_{xx,k} & P_{xy,k} \\ P_{yx,k} & P_{yy,k} \end{bmatrix}. \tag{28}$$

The ellipses are initially large because the initial covariance matrix P_0 assigns high uncertainty to the initial state. As the EKF processes more measurements, the uncertainty ellipses shrink significantly. This indicates that the filter becomes more confident in its position estimate.

In the later part of the trajectory, the ellipses remain relatively small and bounded.

This shows that the EKF reaches a stable estimation regime where the prediction and correction steps balance each other.

5.3 Heading Angle Estimation

The heading angle estimation result is shown in Figure 3.

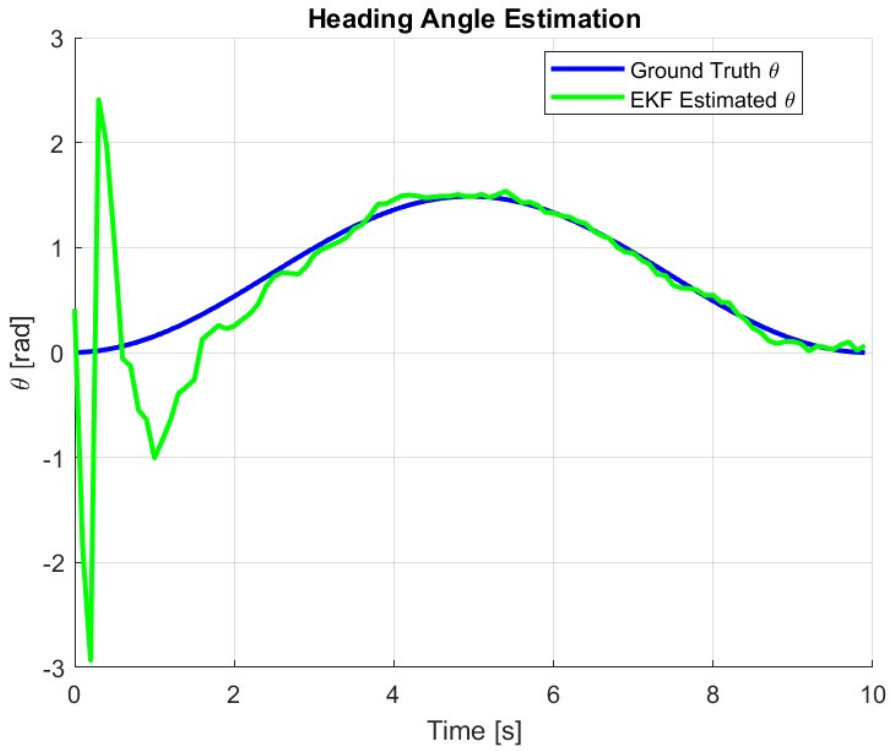


Figure 3: Comparison between ground-truth heading angle and EKF estimated heading angle.

Although the heading angle is not directly measured, the EKF is able to estimate it. This is possible because the heading angle affects the predicted position through the nonlinear motion model:

$$p_{x,k} = p_{x,k-1} + v_k \cos(\theta_{k-1})\Delta t, \tag{29}$$

$$p_{y,k} = p_{y,k-1} + v_k \sin(\theta_{k-1})\Delta t. \tag{30}$$

Therefore, when there is a difference between the predicted position and the measured position, the EKF correction step can also update the heading estimate through the cross-covariance terms in P_k.

A noticeable transient occurs at the beginning of the heading estimation. This is expected because the heading angle is not directly measured and the initial covariance is

large. After this transient period, the EKF heading estimate converges and closely follows the ground-truth heading angle.

5.4 Position Error

The position estimation error is shown in Figure 4. The position error was computed as

$$e_p(k) = \sqrt{(p_{x,k}^{true} - \hat{p}_{x,k})^2 + (p_{y,k}^{true} - \hat{p}_{y,k})^2}. \tag{31}$$

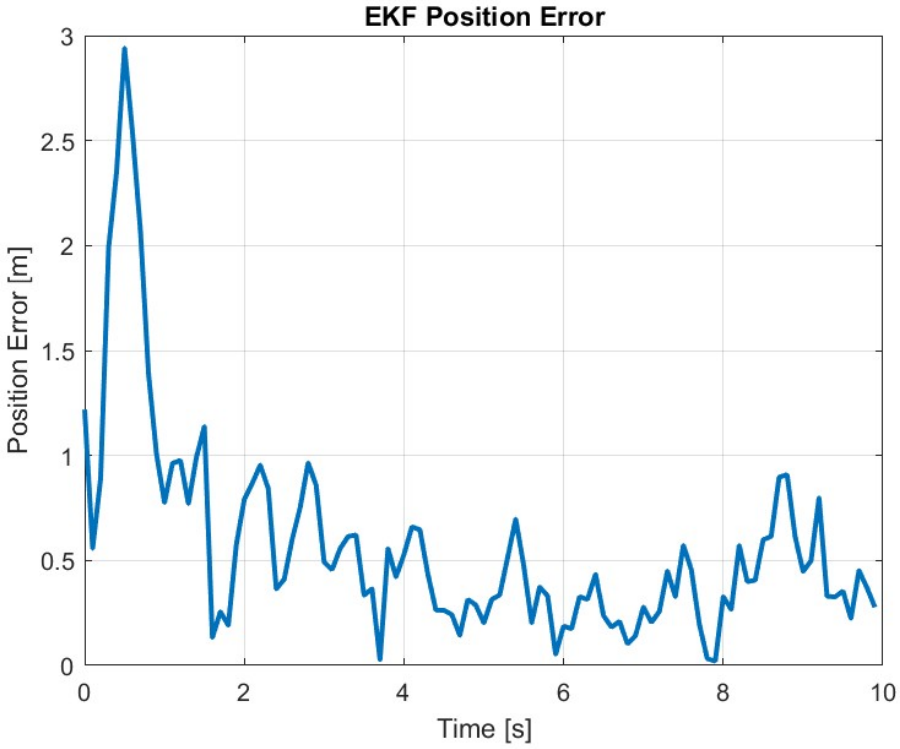


Figure 4: Position estimation error of the EKF.

The position error has a short initial peak due to the mismatch between the initial estimate and the true state. After the first few updates, the error decreases rapidly and remains mostly below approximately 1 m. This confirms that the EKF successfully reduces the measurement noise while tracking the vehicle trajectory.

5.5 Heading Error

The heading estimation error is shown in Figure 5. The heading error was computed using angle wrapping:

$$e_\theta(k) = \operatorname{atan2}\left(\sin(\theta_k^{true} - \hat{\theta}_k), \cos(\theta_k^{true} - \hat{\theta}_k)\right). \tag{32}$$

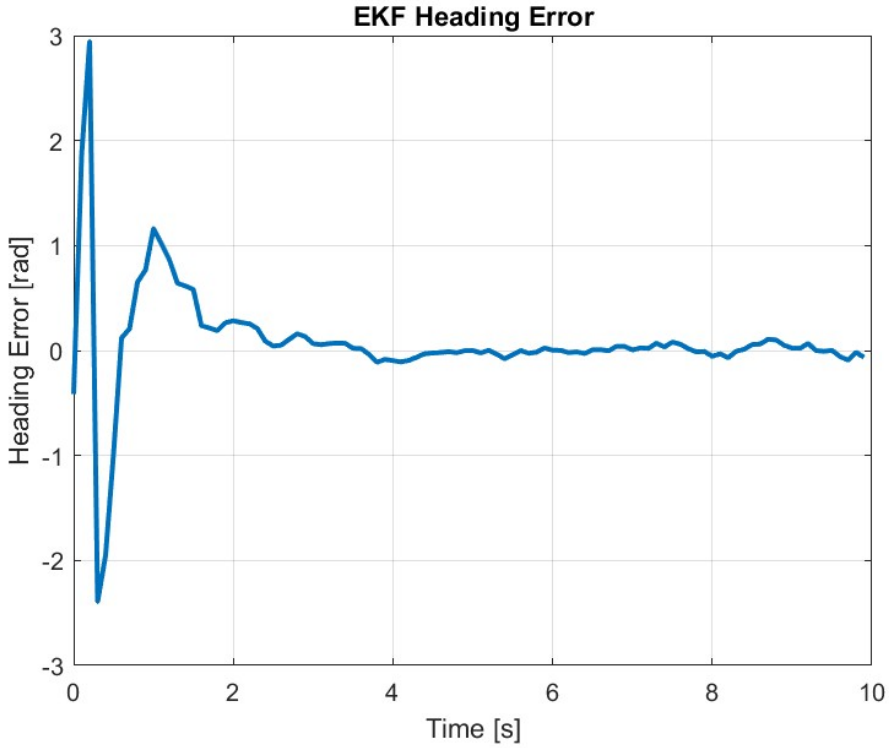


Figure 5: Heading angle estimation error of the EKF.

The heading error is relatively large during the first part of the simulation due to the initial transient. However, after convergence, the error remains close to zero. This indicates that the EKF can estimate the unmeasured heading angle effectively by using the relationship between heading and position evolution in the vehicle model.

5.6 Performance Metrics

The numerical estimation performance is summarized in Table 2.

Table 2: EKF performance metrics for the nominal measurement covariance matrix.

Metric	Value
RMSE of p_x	0.5228 m
RMSE of p_y	0.5601 m
RMSE of position	0.7662 m
RMSE of heading angle θ	0.5372 rad

The position RMSE value of 0.7662 m indicates that the EKF achieves good position tracking performance despite the noisy GPS-like measurements. The heading RMSE is affected by the initial transient. After the transient period, the heading estimate becomes much closer to the ground truth.

6 Effect of Increasing Measurement Noise Covariance

6.1 Motivation

The measurement noise covariance matrix R represents the assumed uncertainty of the measurement sensor. In this case, R models the noise level of the GPS-like position measurements. To analyze the effect of measurement uncertainty, the EKF was tested with an increased measurement covariance matrix:

$$R_{\text{large}} = 50R. \tag{33}$$

Since the nominal matrix was

$$R = \begin{bmatrix} 1.5 & 0 \\ 0 & 1.5 \end{bmatrix}, \tag{34}$$

the increased measurement noise covariance became

$$R_{\text{large}} = \begin{bmatrix} 75 & 0 \\ 0 & 75 \end{bmatrix}. \tag{35}$$

6.2 Theoretical Effect of Increasing R

The Kalman gain is given by

$$K_k = P_k^- H_k^T \left(H_k P_k^- H_k^T + R \right)^{-1}. \tag{36}$$

The term

$$S_k = H_k P_k^- H_k^T + R \tag{37}$$

is the innovation covariance. When R increases, S_k also increases. As a result, the Kalman gain K_k generally decreases. Therefore, the EKF gives less weight to the measurements and relies more strongly on the motion model.

This means that increasing R makes the filter less responsive to noisy measurement fluctuations. However, if the process model is inaccurate, an excessively large R may also prevent the EKF from correcting prediction errors.

6.3 Comparison Results

The trajectory comparison for the nominal and increased measurement covariance cases is shown in Figure 6.

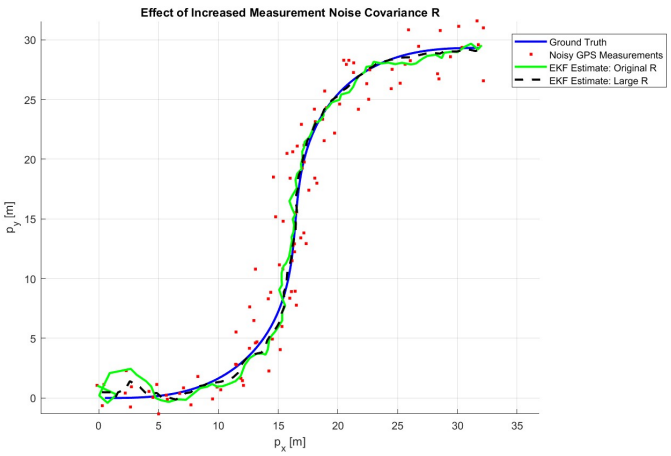


Figure 6: Trajectory comparison between nominal R and increased R.

The corresponding position error comparison is shown in Figure 7.

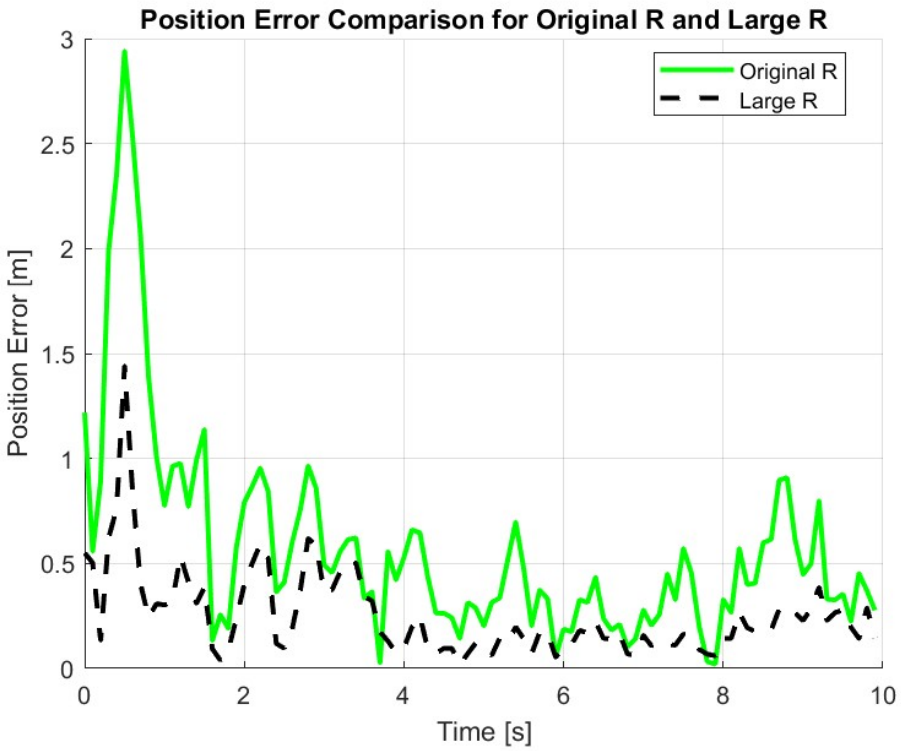


Figure 7: Position error comparison between nominal R and increased R.

The numerical comparison is given in Table 3.

Table 3: Effect of increasing measurement noise covariance R.

Case	Position RMSE
Nominal R	0.7662 m
Large $R = 50R_{\text{nominal}}$	0.3263 m

Interestingly, increasing R decreased the position RMSE from 0.7662 m to 0.3263 m. Although increasing R often reduces estimation performance when the process model is inaccurate, the opposite behavior was observed in this simulation.

This result indicates that, for the given dataset, the kinematic bicycle model is highly consistent with the ground-truth trajectory generation, while the GPS-like measurements contain noticeable noise. By increasing R, the EKF trusts the noisy measurements less and relies more on the accurate motion model. Consequently, the estimated trajectory becomes smoother and closer to the ground truth in terms of position RMSE.

However, this should not be interpreted as a general rule that increasing R always improves EKF performance. In a real system with modeling errors, disturbances, tire slip, actuator uncertainty, or changing vehicle speed, an excessively large R could degrade performance because the filter would ignore useful measurement corrections.

7 Discussion

The results demonstrate that the Extended Kalman Filter is suitable for estimating the state of a vehicle governed by a nonlinear kinematic bicycle model. The filter successfully combines the prediction from the vehicle model with noisy GPS-like measurements.

The trajectory plot shows that the EKF produces a smoother estimate than the raw GPS measurements. This is expected because the EKF does not simply follow each noisy measurement point. Instead, it balances the model prediction and measurement correction according to the covariance matrices Q and R.

The uncertainty ellipses provide a useful visualization of the covariance evolution. The large initial ellipse corresponds to the large initial uncertainty. As measurements are incorporated, the covariance decreases and the ellipses become smaller. This indicates that the EKF becomes more confident in the position estimate.

The heading estimation result is also important. Even though θ is not directly measured, it is estimated through the nonlinear coupling between heading and position. Since the predicted position depends on $\cos(\theta)$ and $\sin(\theta)$, position measurements contain indirect information about the heading angle. The EKF uses the cross-covariance between position and heading to update θ during the correction step.

The increased-R experiment shows the importance of covariance tuning. In this dataset, larger R improved position RMSE because the motion model was more reliable than the noisy GPS measurements. In practical autonomous vehicle applications, however, the best value of R depends on sensor quality, model accuracy, and operating conditions.

8 Conclusion

In this homework, an Extended Kalman Filter was derived, implemented, and analyzed for two-dimensional vehicle motion estimation. The nonlinear process model was based on the kinematic bicycle model, and the measurement model consisted of noisy GPS-like position measurements.

The EKF was implemented in Simulink using separate prediction and update subsystems. The prediction block used the nonlinear vehicle model and the corresponding analytical Jacobian. The update block used position measurements to correct both the position and heading estimates.

The simulation results showed that the EKF estimate closely follows the ground-truth trajectory while filtering measurement noise. The uncertainty ellipses shrink over time, indicating that the filter becomes more confident as measurements are processed. The heading angle, although not directly measured, was successfully estimated through its effect on the vehicle position dynamics.

The nominal EKF achieved a position RMSE of 0.7662 m. When the measurement noise covariance matrix was increased by a factor of 50, the position RMSE decreased to 0.3263 m. This occurred because the process model was highly accurate for the given dataset, and increasing R reduced the influence of noisy GPS measurements. Nevertheless, this improvement is specific to the given simulation and should not be generalized without considering model uncertainty.

Overall, the EKF provided reliable state estimation performance for the given nonlinear vehicle estimation problem.

A MATLAB Initialization Script

```
clear; clc; close all;

% Load original homework data
load("hw2_ekf_data.mat");

% Number of samples
N = size(x_true, 2);

% Time vector
t = (0:N-1)' * dt;

% Initial conditions for Simulink Unit Delay blocks
x0 = x_hat;
P0 = P;
```

```
15
16 % Construct Simulink-compatible input signal
17 % u = [delta, v]
18 u_data = [delta_arr(:), v*ones(N,1)];
19 u_sim = timeseries(u_data, t);
20
21 % Construct Simulink-compatible measurement signal
22 % z = [px_meas, py_meas]
23 z_data = z_meas';
24 z_sim = timeseries(z_data, t);
25
26 disp("EKF initialization completed.")
27 disp("Original .mat file was only loaded, not modified.")
28 disp(["N = ", num2str(N)])
29 disp(["size(u_data) = ", mat2str(size(u_data))])
30 disp(["size(z_data) = ", mat2str(size(z_data))])
```

Listing 1: MATLAB initialization script used for Simulink-compatible input signals.

B Prediction Block MATLAB Function

```
1  function [x_pred, P_pred] = prediction_block(x_prev, P_prev, u,
       dt, L, Q)
2  % EKF prediction step for kinematic bicycle model
3
4  % Make input vector safe
5  u = u(:);
6
7  % States
8  px    = x_prev(1);
9  py    = x_prev(2);
10 theta = x_prev(3);
11
12 % Inputs
13 delta = u(1);
14 v     = u(2);
15
16 % Nonlinear state prediction
17 x_pred = zeros(3,1);
18
19 x_pred(1) = px + v*cos(theta)*dt;
```

```
x_pred(2) = py + v*sin(theta)*dt;
x_pred(3) = theta + (v/L)*tan(delta)*dt;

% Normalize heading angle
x_pred(3) = atan2(sin(x_pred(3)), cos(x_pred(3)));

% Jacobian matrix
F = [1, 0, -v*sin(theta)*dt;
     0, 1,  v*cos(theta)*dt;
     0, 0,  1];

% Covariance prediction
P_pred = F*P_prev*F' + Q;
end
```

Listing 2: MATLAB Function block code for EKF prediction step.

C Update Block MATLAB Function

```
function [x_hat, P] = update_block(x_pred, P_pred, z, R)
% EKF update step for GPS-like position measurement

% Make measurement vector safe
z = z(:);

% Measurement matrix
H = [1, 0, 0;
     0, 1, 0];

% Predicted measurement
z_pred = H*x_pred;

% Innovation
y = z - z_pred;

% Innovation covariance
S = H*P_pred*H' + R;

% Kalman gain
K = (P_pred*H') / S;

```

```
% State update
x_hat = x_pred + K*y;

% Normalize heading angle
x_hat(3) = atan2(sin(x_hat(3)), cos(x_hat(3)));

% Covariance update
I = eye(3);
P = (I - K*H)*P_pred;
end
```

Listing 3: MATLAB Function block code for EKF update step.

D Post-Processing Script

```
%% EKF Post-Processing for Simulink Output Object
clc; close all;

%% Load original data
load("hw2_ekf_data.mat");

%% Extract Simulink logs
t_sim = out.tout;

x_est_raw = out.x_hat_log;
P_raw     = out.P_log;

%% Format x_hat_log
x_est_raw = squeeze(x_est_raw);

if size(x_est_raw,1) == 3
    x_est = x_est_raw;
elseif size(x_est_raw,2) == 3
    x_est = x_est_raw';
else
    error("Unexpected x_hat_log size: %s", mat2str(size(x_est_raw
        )));
end

%% Format P_log
P_raw = squeeze(P_raw);
```

```
26
27 if ndims(P_raw) == 3
28     if size(P_raw,1) == 3 && size(P_raw,2) == 3
29         P_est = P_raw;
30     elseif size(P_raw,2) == 3 && size(P_raw,3) == 3
31         P_est = permute(P_raw, [2 3 1]);
32     else
33         error("Unexpected P_log 3D size: %s", mat2str(size(P_raw)
               ));
34     end
35 elseif ismatrix(P_raw)
36     if size(P_raw,2) == 9
37         P_est = zeros(3,3,size(P_raw,1));
38         for k = 1:size(P_raw,1)
39             P_est(:,:,k) = reshape(P_raw(k,:), [3,3]);
40         end
41     elseif size(P_raw,1) == 9
42         P_est = zeros(3,3,size(P_raw,2));
43         for k = 1:size(P_raw,2)
44             P_est(:,:,k) = reshape(P_raw(:,k), [3,3]);
45         end
46     else
47         error("Unexpected P_log matrix size: %s", mat2str(size(
               P_raw)));
48     end
49 else
50     error("Unsupported P_log format.");
51 end
52
53 %% Match lengths
54 N_true = size(x_true, 2);
55 N_est  = size(x_est, 2);
56 N_cov  = size(P_est, 3);
57 N_time = length(t_sim);
58
59 N = min([N_true, N_est, N_cov, N_time]);
60
61 x_true_plot = x_true(:,1:N);
62 z_meas_plot = z_meas(:,1:N);
63 x_est_plot  = x_est(:,1:N);
64 P_plot      = P_est(:,:,1:N);
```

```
t_plot      = t_sim(1:N);

%% 2D Trajectory Plot with Uncertainty Ellipses
figure;
hold on; grid on; axis equal;

plot(x_true_plot(1,:), x_true_plot(2,:), 'b-', 'LineWidth', 2);
plot(z_meas_plot(1,:), z_meas_plot(2,:), 'r.', 'MarkerSize', 10);
plot(x_est_plot(1,:), x_est_plot(2,:), 'g-', 'LineWidth', 2);

ellipse_indices = round(linspace(1, N, 8));

for idx = ellipse_indices
    mu = x_est_plot(1:2, idx);
    Sigma = P_plot(1:2,1:2,idx);
    plot_uncertainty_ellipse(mu, Sigma, 2.0);
end

xlabel('p_x [m]');
ylabel('p_y [m]');
title('2D Trajectory with EKF Estimate and Uncertainty Ellipses')
   ;
legend('Ground Truth', 'Noisy GPS Measurements', 'EKF Estimate',
   ...
       'Uncertainty Ellipses', 'Location', 'best');

%% Heading Angle Plot
figure;
hold on; grid on;

plot(t_plot, x_true_plot(3,:), 'b-', 'LineWidth', 2);
plot(t_plot, x_est_plot(3,:), 'g-', 'LineWidth', 2);

xlabel('Time [s]');
ylabel('\theta [rad]');
title('Heading Angle Estimation');
legend('Ground Truth \theta', 'EKF Estimated \theta', 'Location',
    'best');

%% Position and heading error plots
pos_error = sqrt((x_true_plot(1,:) - x_est_plot(1,:)).^2 + ...
```

```
                (x_true_plot(2,:) - x_est_plot(2,:)).^2);

theta_error = wrapToPiLocal(x_true_plot(3,:) - x_est_plot(3,:));

figure;
plot(t_plot, pos_error, 'LineWidth', 2);
grid on;
xlabel('Time [s]');
ylabel('Position Error [m]');
title('EKF Position Error');

figure;
plot(t_plot, theta_error, 'LineWidth', 2);
grid on;
xlabel('Time [s]');
ylabel('Heading Error [rad]');
title('EKF Heading Error');

%% Numerical performance metrics
rmse_px = sqrt(mean((x_true_plot(1,:) - x_est_plot(1,:)).^2));
rmse_py = sqrt(mean((x_true_plot(2,:) - x_est_plot(2,:)).^2));
rmse_pos = sqrt(mean(pos_error.^2));
rmse_theta = sqrt(mean(theta_error.^2));

fprintf("\n===== EKF PERFORMANCE METRICS =====\n");
fprintf("RMSE px       = %.4f m\n", rmse_px);
fprintf("RMSE py       = %.4f m\n", rmse_py);
fprintf("RMSE position = %.4f m\n", rmse_pos);
fprintf("RMSE theta    = %.4f rad\n", rmse_theta);

%% Local functions
function plot_uncertainty_ellipse(mu, Sigma, n_sigma)
    Sigma = (Sigma + Sigma')/2;

    [V,D] = eig(Sigma);

    angle = linspace(0, 2*pi, 100);
    circle = [cos(angle); sin(angle)];

    ellipse = mu(:) + n_sigma * V * sqrt(max(D,0)) * circle;

```

```
    plot(ellipse(1,:), ellipse(2,:), 'm-', 'LineWidth', 1.5);
end

function ang = wrapToPiLocal(ang)
    ang = atan2(sin(ang), cos(ang));
end
```

Listing 4: MATLAB post-processing script for EKF plots and metrics.

E Measurement Noise Covariance Comparison Script

```
ekf_init

R_original = R;

% Normal case
R = R_original;
out_normal = sim("ekf_hw2");

% Increased measurement noise covariance case
R = 50 * R_original;
out_largeR = sim("ekf_hw2");

% Restore original R
R = R_original;

%% Compare normal R and large R
load("hw2_ekf_data.mat");

t_normal = out_normal.tout;
x_normal = squeeze(out_normal.x_hat_log);

t_largeR = out_largeR.tout;
x_largeR = squeeze(out_largeR.x_hat_log);

% Fix orientation
if size(x_normal,2) == 3
    x_normal = x_normal';
end

if size(x_largeR,2) == 3
```

```
    x_largeR = x_largeR';
end

N = min([size(x_true,2), size(x_normal,2), size(x_largeR,2), ...
    length(t_normal)]);

x_true_c = x_true(:,1:N);
z_meas_c = z_meas(:,1:N);
x_normal = x_normal(:,1:N);
x_largeR = x_largeR(:,1:N);
t_c = t_normal(1:N);

figure;
hold on; grid on; axis equal;

plot(x_true_c(1,:), x_true_c(2,:), 'b-', 'LineWidth', 2);
plot(z_meas_c(1,:), z_meas_c(2,:), 'r.', 'MarkerSize', 8);
plot(x_normal(1,:), x_normal(2,:), 'g-', 'LineWidth', 2);
plot(x_largeR(1,:), x_largeR(2,:), 'k--', 'LineWidth', 2);

xlabel('p_x [m]');
ylabel('p_y [m]');
title('Effect of Increased Measurement Noise Covariance R');
legend('Ground Truth', 'Noisy GPS Measurements', ...
       'EKF Estimate: Original R', 'EKF Estimate: Large R', ...
       'Location', 'best');

% Error comparison
pos_error_normal = sqrt((x_true_c(1,:) - x_normal(1,:)).^2 + ...
                        (x_true_c(2,:) - x_normal(2,:)).^2);

pos_error_largeR = sqrt((x_true_c(1,:) - x_largeR(1,:)).^2 + ...
                        (x_true_c(2,:) - x_largeR(2,:)).^2);

figure;
hold on; grid on;

plot(t_c, pos_error_normal, 'g-', 'LineWidth', 2);
plot(t_c, pos_error_largeR, 'k--', 'LineWidth', 2);

xlabel('Time [s]');
```

```
ylabel('Position Error [m]');
title('Position Error Comparison for Original R and Large R');
legend('Original R', 'Large R', 'Location', 'best');

rmse_pos_normal = sqrt(mean(pos_error_normal.^2));
rmse_pos_largeR = sqrt(mean(pos_error_largeR.^2));

fprintf("\n===== R COMPARISON =====\n");
fprintf("RMSE position, original R = %.4f m\n", rmse_pos_normal);
fprintf("RMSE position, large R    = %.4f m\n", rmse_pos_largeR);
```

Listing 5: MATLAB script for comparing nominal and increased measurement noise covariance.